@azdocuments

PYTHON PROGRAMMING

SUBJECT CODE: 1BPLC105B/205B

MODULE-5

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

PYTHON PROGRAMMING

1BPLC105B/205B

Module-5

Object oriented programming: Objects are mutable, Sameness, Copying.

Inheritance: Pure functions ,Modifiers, Generalization, Operator Overloading, Polymorphism.

Exceptions: Catching Exceptions, Raising your own exceptions.

Chapter: 11.2.2-11.2.4, 11.3.2-11.3.9, 12.1, 12.2

5.1.1 Objects Are Mutable

In Python, **objects created from user-defined classes are mutable by default**.
This means the **state of an object can be changed after it has been created** by modifying its attributes.

Changing Object State via Attributes

An object's **state** is stored in its attributes.
We can change this state by assigning new values to those attributes.

Example (Conceptual)

box.width += 50

box.height += 100

- The object box remains the **same object**
- Only its **internal state** (width and height) changes

Encapsulation Using Methods (Preferred Approach)

Rather than modifying attributes directly, it is better practice to **encapsulate changes inside methods**.
This keeps object behavior:

- Organized
- Safer
- Easier to maintain

Rectangle Class with Mutable Behavior

class Rectangle:

```
    # ...
def grow(self, delta_width, delta_height):
        """ Grow (or shrink) this object by the deltas """
        self.width += delta_width
        self.height += delta_height
def move(self, dx, dy):
        """ Move this object by the deltas """
        self.corner.x += dx
        self.corner.y += dy
```

Understanding the Methods

grow(delta_width, delta_height)

- Changes the **size** of the rectangle
- Can:
    - Increase size (positive values)
    - Decrease size (negative values)
- Does **not** change position

move(dx, dy)

- Changes the **position** of the rectangle
- Updates the coordinates of its corner
- The size remains unchanged

Using the Rectangle Object

```
r = Rectangle(Point(10, 5), 100, 50)
print(r)
```

Output

```
((10, 5), 100, 50)
```

- Corner at (10, 5)
- Width = 100
- Height = 50

Growing the Rectangle

r.grow(25, -10)

print(r)

Output

((10, 5), 125, 40)

- Width increased by 25
- Height decreased by 10
- Position unchanged

Moving the Rectangle

r.move(-10, 10)

print(r)

Output

((0, 15), 125, 40)

- Rectangle moved left by 10
- Rectangle moved up by 10
- Size unchanged
- The object r is **not replaced**
- The same object is updated internally
- Methods modify the object's **existing state**
- This demonstrates **object mutability**
- Objects are **mutable by default**
- Object state is stored in **attributes**
- Attributes can be changed after object creation
- Methods are used to **encapsulate state changes**
- Mutability allows objects to model real-world behavior
- One object can change over time while keeping its identity

5.1.2 Sameness

The idea of two things being the **"same"** can mean different things depending on context. This distinction is very important when working with **objects in Python**.

Two Meanings of "Same" (Real-World Analogy)

- **Same kind, different object**
  "Alice and Bob have the same car" → same make and model, but **two different cars**.
- **Exactly the same object**
  "Alice and Bob have the same mother" → **one single person**.

Objects in Python have the same ambiguity.

Sameness in Python Objects

When comparing two objects, we must ask:

1. Are they the **same object in memory**?
2. Or do they just contain the **same data**?

Shallow Equality (Identity)

Using the is Operator

The is operator checks whether **two variables refer to the same object**.

p1 = Point(3, 4)

p2 = Point(3, 4)

p1 is p2

Output

False

- p1 and p2 have the same coordinates
- But they are **different objects in memory**

Aliasing Example

p3 = p1

p1 is p3

Output

True

- p1 and p3 refer to the **same object**
- This is called **aliasing**

This type of comparison is called **shallow equality** (identity equality).

Deep Equality (Content Equality)

Deep equality compares the **contents** of objects, not their memory location.

Custom Function for Deep Equality

```
def same_coordinates(p1, p2):
    return (p1.x == p2.x) and (p1.y == p2.y)
```

Example

```
p1 = Point(3, 4)
p2 = Point(3, 4)
same_coordinates(p1, p2)
```

Output

True

- Different objects
- Same data
- Therefore, **deeply equal**

Relationship Between Shallow and Deep Equality

- If two variables refer to the **same object**:
  - They are **shallowly equal**
  - They are also **deeply equal**
- If two variables refer to **different objects**:
  - They may be deeply equal
  - But they are not shallowly equal

Beware of == (Very Important)

Python allows class designers to decide what == means for their objects.

Example Comparison

```
p = Point(4, 2)
s = Point(4, 2)
print(p == s)
```

Output

False

- By default, == on user-defined objects like Point
- Performs **shallow equality** (identity comparison)

Contrast with Lists

```
a = [2, 3]
```

```
b = [2, 3]
print(a == b)
```

Output

True

- Lists use **deep equality** by default
- Contents are compared element by element

Type	Comparison	Meaning
is	Shallow	Same object in memory
Custom function	Deep	Same contents
== (Point)	Shallow (default)	Same object
== (List)	Deep (default)	Same values

- "Sameness" can mean **same object** or **same data**
- is checks **identity**, not contents
- Aliasing occurs when two variables refer to the same object
- Deep equality compares attribute values
- == behavior depends on the class implementation
- Built-in collections like lists use deep equality by default
- User-defined classes use shallow equality unless overridden

5.1.3 Copying

Aliasing occurs when multiple variables refer to the **same object**. This can make programs hard to understand and debug, because a change made through one reference can unexpectedly affect another. A common way to avoid aliasing is to **copy objects** so that each variable refers to its **own independent object**.

The copy Module

Python provides the **copy module**, which supports object duplication.

```
import copy
```

Shallow Copy

A **shallow copy** duplicates the object itself, but **does not duplicate objects it refers to**.

Example with a Simple Object (Point)

import copy

p1 = Point(3, 4)

p2 = copy.copy(p1)

p1 is p2

Output

False

same_coordinates(p1, p2)

Output

True

Explanation

- p1 and p2 are **different objects** (no aliasing)
- They contain the **same data**
- For simple objects (no embedded objects), **shallow copy is sufficient**

Why Shallow Copy Can Be a Problem

Consider a more complex object like **Rectangle**, which contains a **reference to a Point** (e.g., the corner).

When we do:

b2 = copy.copy(b1)

- The Rectangle object is copied
- But the **embedded Point object is not copied**
- Both rectangles now refer to the **same Point**

Consequences

- Calling grow() on one rectangle:
  - Changes only that rectangle's size
- Calling move() on one rectangle:
  - Changes the shared Point
  - Affects **both rectangles**

This behavior is:

- Confusing
- Error-prone
- Usually **not what we want**

Deep Copy

A **deep copy** duplicates:

- The object
- **All objects referenced by it (recursively)**

This ensures **complete independence**.

Using deepcopy

b2 = copy.deepcopy(b1)

Result

- b1 and b2 are **completely separate objects**
- Their embedded Point objects are also separate
- Changes to one object **do not affect** the other

Feature	Shallow Copy (copy.copy)	Deep Copy (copy.deepcopy)
Copies main object	Yes	Yes
Copies embedded objects	No	Yes
Risk of aliasing	Yes	No
Suitable for simple objects	Yes	Yes
Suitable for nested objects	No	Yes

- Aliasing can cause unexpected side effects
- Copying is an alternative to aliasing
- copy.copy() creates a **shallow copy**
- Shallow copy duplicates references to embedded objects
- copy.deepcopy() creates a **deep copy**
- Deep copy duplicates all nested objects
- Deep copy avoids shared state and bugs

5.2 Pure Functions

In this section, we study **pure functions** by writing a function add_time that adds two MyTime objects. This prepares us to contrast pure functions with **modifier functions** later.

What Is a Pure Function?

A **pure function** has the following properties:

1. **Does not modify** any of its input objects
2. **Has no side effects**, such as:
    - Changing global variables
    - Printing output
    - Taking user input
3. Returns a **new result** based only on its inputs

Because of these properties, pure functions are:

- Easier to understand
- Easier to test
- Safer to reuse

Initial Version of add_time

```
def add_time(t1, t2):
    h = t1.hours + t2.hours
    m = t1.minutes + t2.minutes
    s = t1.seconds + t2.seconds
    sum_t = MyTime(h, m, s)
    return sum_t
```

Explanation

- The function reads values from t1 and t2
- It creates a **new MyTime object**
- It returns the new object
- The original objects are **not changed**

Hence, this function is **pure**.

Using the Pure Function

```
current_time = MyTime(9, 14, 30)
bread_time = MyTime(3, 35, 0)
done_time = add_time(current_time, bread_time)
print(done_time)
```

Output

12:49:30

This result is correct for this input.

The Hidden Problem

The function fails when:

- Seconds add up to **60 or more**
- Minutes add up to **60 or more**

Example:

MyTime(10, 50, 40) + MyTime(1, 15, 30)

- Seconds = 40 + 30 = 70
- Minutes = 50 + 15 = 65

The function does **not handle carry-over**.

Improved Version with Carry Handling

```
def add_time(t1, t2):
 h = t1.hours + t2.hours
    m = t1.minutes + t2.minutes
    s = t1.seconds + t2.seconds
 if s >= 60:
        s -= 60
        m += 1
   if m >= 60:
        m -= 60
        h += 1
```

```
sum_t = MyTime(h, m, s)
  return sum_t
```

What This Version Fixes

- Handles:
  - Carry from seconds → minutes
  - Carry from minutes → hours
- Still a **pure function**
- Still returns a **new object**
- Still does **not modify inputs**

Remaining Limitation

Even this improved version:

- Handles only **one level of carry**
- Does not correctly handle cases like:
  - Very large seconds or minutes
  - Multiple carries

The function is becoming **longer and more complex**, which motivates a better design approach later.

Pure functions:

- Are correct in **design philosophy**
- Avoid unintended side effects
- But may require **careful logic** to handle all edge cases

Later improvements will aim to:

- Simplify the logic
- Improve correctness
- Maintain purity
- A pure function does **not modify its arguments**
- It has **no side effects**
- It returns a **new object**

- add_time is a pure function
- Carry-over logic is necessary for time calculations
- Pure functions are easier to test and reason about
- Increasing complexity suggests the need for better abstraction

5.2.1 Modifiers

In contrast to **pure functions**, which return new objects and leave their inputs unchanged, **modifier functions** are designed to **change (modify) the state of existing objects** passed to them as parameters.

Because the caller usually keeps a reference to the object, **any modification made inside the function is visible outside the function** as well.

What Is a Modifier?

A **modifier**:

- Takes one or more objects as parameters
- **Changes the internal state** of those objects
- Usually does **not return a new object**
- Has a **side effect** (the object is altered)

Example: increment as a Modifier

Suppose we want to add a number of seconds to a MyTime object. The most natural design is to **modify the object itself**.

Initial Version (Rough Draft)

```
def increment(t, secs):
    t.seconds += secs
 if t.seconds >= 60:
        t.seconds -= 60
        t.minutes += 1
   if t.minutes >= 60:
        t.minutes -= 60
        t.hours += 1
```

Explanation

- The function directly updates t.seconds, t.minutes, and t.hours

- The object t is changed in place
- The caller sees the updated time

Problem with the Initial Version

This function is **not correct in all cases**.

What Goes Wrong?

- If secs is **much greater than 60**, a single if statement is not enough
- Example:
    - Adding 3600 seconds
    - Requires **multiple carry operations**

The function only carries **once**, which leads to incorrect results.

Improved Version Using while

To handle large values of seconds correctly, we replace if with while.

```
def increment(t, seconds):
    t.seconds += seconds
while t.seconds >= 60:
        t.seconds -= 60
        t.minutes += 1
   while t.minutes >= 60:
        t.minutes -= 60
        t.hours += 1
```

What This Version Fixes

- Handles **any number of seconds ≥ 0**
- Repeatedly carries:
    - Seconds → Minutes
    - Minutes → Hours
- Correctly modifies the object's state

Remaining Limitations

Even though this version is logically correct:

- It assumes seconds is **not negative**
- It assumes hours does not exceed **23**
- The repeated while loops make the code:
    - Longer
    - Less elegant
    - Harder to maintain

This shows that **correctness alone does not guarantee good design**.

Pure Functions vs Modifiers (Quick Comparison)

Aspect	Pure Function	Modifier
Changes input objects	No	Yes
Returns a new object	Yes	Usually No
Side effects	None	Yes
Easier to test	Yes	Harder
Example	add_time	increment

- Modifiers **change the state** of objects
- Changes are visible to the caller due to shared references
- increment is a modifier function
- while loops are needed for repeated carry operations
- Modifiers have **side effects**
- Correct logic does not always imply **good design**

5.2.2 Converting increment to a Method

In object-oriented programming, it is considered **better design** to place functions that operate on an object **inside the class of that object**. Since increment works on MyTime objects, OOP style suggests converting it from a standalone function into a **method of the MyTime class**.

From Function to Method (Conceptual Change)

Earlier, increment was written as a **modifier function**:

increment(t, seconds)

This meant:

- The function was external
- The object was passed in as a parameter

In OOP style, we prefer:

t.increment(seconds)

This means:

- The object is the **active agent**
- The behavior belongs to the object itself

increment as a Method in MyTime

```
class MyTime:
    # Previous method definitions here...
 def increment(self, seconds):
        self.seconds += seconds
while self.seconds >= 60:
            self.seconds -= 60
            self.minutes += 1
while self.minutes >= 60:
            self.minutes -= 60
            self.hours += 1
```

What Changed?

The transformation is **purely mechanical**:

1. The function definition is moved **inside the class**
2. The first parameter is renamed to self
3. Attribute access uses self.hours, self.minutes, self.seconds
4. The logic of the function remains the same

How the Method Is Called

current_time.increment(500)

What Happens Internally

- current_time is automatically passed as the first argument (self)
- seconds receives the value 500
- The method modifies the **state of current_time**

This is equivalent to:

increment(current_time, 500)

but follows **object-oriented style**.

Why This Is Better OOP Design

- Keeps **data and behavior together**
- Improves readability and maintainability
- Matches real-world thinking:
  - Objects perform actions on themselves
- Reduces the need for external helper functions

The example omits previously defined methods **only to save space**.
In a complete implementation, the MyTime class should also include:

- Initializer (__init__)
- String conversion (__str__)
- Other time-related methods
- Functions operating on an object should be placed inside its class
- Converting a function to a method is mostly a **mechanical change**
- self refers to the object invoking the method
- Method calls use dot notation: object.method()
- The method modifies the object's internal state
- This follows proper **object-oriented design principles**

5.2.3 An "Aha!" Insight

Sometimes a **high-level insight** can dramatically simplify a programming problem.
In this case, the key insight is:

A MyTime object is essentially a three-digit number in base 60.

Understanding Time as Base-60

A time value consists of:

- **Seconds** → ones column (base 60)
- **Minutes** → sixties column
- **Hours** → thirty-six-hundreds column (60 × 60)

So when we previously wrote add_time and increment, we were actually:

- Performing **addition in base 60**
- Manually handling **carry-over** between seconds → minutes → hours

That is why the earlier solutions required multiple if or while statements.

New Approach: Convert Time to Seconds

Instead of working directly with hours, minutes, and seconds, we can:

1. Convert a MyTime object into a **single integer value (seconds)**
2. Perform normal arithmetic
3. Convert the result back into a MyTime object

Computers are very good at arithmetic with integers—so let them do the hard work.

Converting a MyTime Object to Seconds

Add this method to the MyTime class:

```
class MyTime:
    # ...
def to_seconds(self):
        """ Return the number of seconds represented
        by this instance
        """
        return self.hours * 3600 + self.minutes * 60 + self.seconds
```

This method:

- Has **no side effects**
- Returns a simple integer
- Makes time arithmetic much easier

Converting Seconds Back to Time

Given a total number of seconds tsecs, we can recover hours, minutes, and seconds using integer arithmetic:

hrs = tsecs // 3600

leftoversecs = tsecs % 3600

mins = leftoversecs // 60

secs = leftoversecs % 60

This works because:

- // gives the whole number of units
- % gives the remainder
- This is a standard **base-conversion technique**

Normalized vs Unnormalized Time

- **Normalized time**:
  3 hours, 12 minutes, 20 seconds
- **Unnormalized time** (same moment):
  2 hours, 70 minutes, 140 seconds

A robust MyTime class should:

- Accept unnormalized input
- Automatically convert it into normalized form

Improved Initializer for MyTime

We redesign the initializer to handle normalization automatically:

```
class MyTime:
    # ...
def __init__(self, hrs=0, mins=0, secs=0):
        """ Create a new MyTime object initialized to hrs, mins, secs.
        The values of mins and secs may be outside the range 0-59,
        but the resulting MyTime object will be normalized.
        """
totalsecs = hrs * 3600 + mins * 60 + secs
        self.hours = totalsecs // 3600
```

```
    leftoversecs = totalsecs % 3600
    self.minutes = leftoversecs // 60
    self.seconds = leftoversecs % 60
```

Benefits

- Handles **any valid integer inputs**
- Guarantees normalized time
- Centralizes the logic in one place

Rewriting add_time (Much Simpler)

Using the new design:

```
def add_time(t1, t2):
    secs = t1.to_seconds() + t2.to_seconds()
    return MyTime(0, 0, secs)
```

Why This Version Is Better

- **Shorter** and cleaner
- Easier to **prove correctness**
- Avoids complex carry logic
- Reuses existing methods
- Demonstrates good **object-oriented design**

This is the power of the “aha!” insight:
by changing perspective, the problem becomes much simpler.

- A MyTime object can be viewed as a **base-60 number**
- Manual carry logic complicates code
- Converting to total seconds simplifies arithmetic
- Integer division and modulo recover hours, minutes, seconds
- A robust initializer should normalize time automatically
- The redesigned add_time is shorter, clearer, and correct
- High-level insight often leads to **better program design**

5.2.4 Generalization

This section explains an important idea in problem solving and program design:
sometimes making a problem more general actually makes it easier to solve.

From Special Case to General Idea

At first glance:

- Working directly with **hours, minutes, and seconds** feels easier
- Base conversion (base 60 → base 10 → base 60) feels more abstract

However, once we **generalize** time into a single integer value (total seconds):

- The code becomes **shorter**
- The logic becomes **simpler**
- The program becomes **more reliable**

This is because:

- We reduce special cases
- We rely on well-understood integer arithmetic

Why Generalization Helps

By converting time to total seconds:

- We avoid manual carry and borrow logic
- We remove repeated conditional checks
- We reduce chances of bugs

Although base conversion is more abstract, it:

- Centralizes complexity in one place
- Makes the rest of the code cleaner

Easier Feature Expansion

Generalization also makes it **easier to add new features**.

Example: Subtracting Two Times

- **Naive approach**:
    - Implement borrowing from minutes and hours
    - Handle many edge cases

- **Generalized approach**:
    - Convert both times to seconds
    - Subtract integers
    - Convert back to MyTime

This approach is:

- Simpler
- Easier to verify
- Less error-prone

The Irony

Sometimes making a problem harder (more abstract) makes the programming easier.

This happens because:

- There are fewer special cases
- There are fewer opportunities for mistakes

Specialization vs Generalization

This section contrasts how **mathematicians** and **computer scientists** often think.

Mathematician's View (Generalization)

Mathematicians often:

- Reduce problems to **integers**
- Treat many objects as **enumerable**

Examples:

- Playing cards → numbers 0 to 51
- Days of the century → numbered sequentially
- Dominoes → indexed values

Their strategy:

- Reduce everything to integers
- Use powerful mathematical tools already known

Computer Scientist's View (Specialization)

Computer scientists often:

- Create **specialized types**
- Restrict meaningless operations

Example:

- Adding two dominoes is meaningless
- Adding two days of the week is meaningless

So instead of using raw integers, we:

- Define types like MyTime
- Control which operations are allowed
- Bundle **data and methods** together

Object-oriented programming is especially good at this.

Combining Both Perspectives

The most effective approach is often to:

- **Think like a mathematician** internally (generalize)
- **Design like a computer scientist** externally (specialize)

In other words:

- Use integers and general logic behind the scenes
- Expose clean, meaningful object-oriented interfaces

- Generalization can reduce complexity
- Base conversion simplifies time arithmetic
- Abstract solutions often reduce special cases
- Generalization improves reliability and extensibility
- Mathematicians favor generalization (integers)
- Computer scientists favor specialization (custom types)
- OOP supports specialization by bundling data and methods
- Good design often balances both approaches

5.2.5 Another Example – Comparing Times

In this section, we look at a function called **after**, whose job is to compare **two MyTime objects** and determine whether the first time is **strictly after** the second time.

Example Use Case

t1 = MyTime(10, 55, 12)

t2 = MyTime(10, 48, 22)

after(t1, t2)

Output

True

This means t1 occurs later than t2.

Writing after as a Method (OOP Style)

Although the function operates on **two objects**, object-oriented design prefers making it a **method of one object**, with the other passed as a parameter.

Initial Method-Based Solution (Field-by-Field Comparison)

```
class MyTime:
    # Previous method definitions here...
def after(self, time2):
        """ Return True if I am strictly greater than time2 """
        if self.hours > time2.hours:
            return True
        if self.hours < time2.hours:
            return False
 if self.minutes > time2.minutes:
            return True
        if self.minutes < time2.minutes:
            return False
if self.seconds > time2.seconds:
            return True
 return False
```

How This Logic Works

- **Hours are compared first**
  - If hours differ, the decision is made immediately
- **Minutes are compared only if hours are equal**
- **Seconds are compared only if both hours and minutes are equal**

This ensures a correct chronological comparison.

Method Invocation

```
if current_time.after(done_time):
    print("The bread will be done before it starts!")
```

This reads almost like English:

If the current time is after the done time…

This shows the expressive power of **object-oriented method calls**.

Applying the "Aha!" Insight (Much Better Solution)

From earlier sections, we learned that a MyTime object can be converted into a **single integer (total seconds)**.

Instead of multiple comparisons, we can:

1. Convert both times to seconds
2. Compare the integers

Improved Version Using to_seconds()

```
class MyTime:
    # Previous method definitions here...
def after(self, time2):
        """ Return True if I am strictly greater than time2 """
        return self.to_seconds() > time2.to_seconds()
```

Why This Version Is Superior

- Much **shorter**
- Much **clearer**

- Easier to **prove correctness**
- Avoids complex conditional logic
- Reuses existing functionality (to_seconds())

This is a textbook example of how a **good abstraction** simplifies code.

- The after method compares two MyTime objects
- OOP style places the method on the **first operand**
- Field-by-field comparison works but is verbose
- Converting time to seconds simplifies comparison
- Integer comparison is clearer and less error-prone
- The "Aha!" insight leads to cleaner, better design

5.2.6 Operator Overloading

Operator overloading allows the **same operator** (such as +, -, *) to have **different meanings for different data types**.

Python already uses operator overloading:

- + adds integers
- + concatenates strings
- * multiplies numbers
- * repeats strings

Python also allows **user-defined classes** to overload operators, which makes objects behave more naturally.

Why Operator Overloading Is Useful

- Makes code **more readable and expressive**
- Allows objects to behave like built-in types
- Supports natural mathematical notation
- Improves object-oriented design

Overloading the + Operator for MyTime

To overload +, we define a special method called __add__.

```
class MyTime:
    # Previously defined methods here...
```

```
def __add__(self, other):
    return MyTime(0, 0, self.to_seconds() + other.to_seconds())
```

Explanation

- self → left operand (t1)
- other → right operand (t2)
- Returns a **new MyTime object**
- Original objects are unchanged (pure behavior)

Using the Overloaded + Operator

```
t1 = MyTime(1, 15, 42)
t2 = MyTime(3, 50, 30)
t3 = t1 + t2
print(t3)
```

Output

```
05:06:12
```

Internally:

t1 + t2 ≡ t1.__add__(t2)

But t1 + t2 is **far more elegant and readable**.

Exercise Idea (As per Textbook)

You can similarly overload subtraction by defining:

```
def __sub__(self, other):
    return MyTime(0, 0, self.to_seconds() - other.to_seconds())
```

Operator Overloading for the Point Class

Overloading + (Point Addition)

```
class Point:
    # Previously defined methods here...
def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
```

This allows:

p1 = Point(3, 4)

p2 = Point(5, 7)

p3 = p1 + p2

Result:

(8, 11)

Overloading the * Operator

Multiplication can behave differently depending on operand order.

Case 1: Point × Point → Dot Product (__mul__)

```
def __mul__(self, other):
    return self.x * other.x + self.y * other.y
```

Example:

```
p1 = Point(3, 4)
p2 = Point(5, 7)
print(p1 * p2)
```

Output

43

This is the **dot product**:

3×5 + 4×7 = 15 + 28 = 43

Case 2: Scalar × Point → Scalar Multiplication (__rmul__)

```
def __rmul__(self, other):
    return Point(other * self.x, other * self.y)
```

Example:

```
print(2 * p2)
```

Output

(10, 14)

Why p2 * 2 Fails

print(p2 * 2)

Error

AttributeError: 'int' object has no attribute 'x'

Explanation

- Left operand is Point
- Python calls __mul__
- __mul__ assumes other is a Point
- But 2 has no .x or .y

Python does **not** automatically switch to __rmul__ here.

Operator overloading:

- Is powerful
- But must be designed carefully
- Requires clear assumptions about operand types
- Can produce confusing errors if misused

This highlights a real challenge in object-oriented programming:

Sometimes it is hard to know **which code is being executed**.

- Operator overloading allows operators to work with user-defined types
- Special methods like __add__, __sub__, __mul__, __rmul__ are used
- a + b → calls a.__add__(b)
- a * b → calls a.__mul__(b)
- b * a → may call a.__rmul__(b)
- Overloading improves readability and expressiveness
- Poorly designed overloading can cause confusing errors

5.2.7 Polymorphism

Polymorphism means "many forms." In programming, it refers to the ability of **the same operation or function to work with different data types**.

In object-oriented programming, polymorphism allows us to write **general-purpose functions** that can operate on many kinds of objects—as long as those objects support the required operations.

Why Polymorphism Is Useful

- Promotes **code reuse**
- Reduces duplication
- Makes programs more flexible and extensible
- Allows new types to work with existing code **without modification**

Example 1: A Polymorphic Function (multadd)

The multadd function multiplies the first two arguments and adds the third:

```
def multadd(x, y, z):
    return x * y + z
```

This function does not care about the **specific types** of x, y, and z.
It only requires that:

- x * y is valid
- (x * y) + z is valid

Using multadd with Numbers

```
multadd(3, 2, 1)
```

Output

```
7
```

Using multadd with Point Objects

```
p1 = Point(3, 4)
p2 = Point(5, 7)
print(multadd(2, p1, p2))
```

Output

(11, 15)

Explanation:

- 2 * p1 → scalar multiplication → (6, 8)
- (6, 8) + p2 → point addition → (11, 15)

print(multadd(p1, p2, 1))

Output

44

Explanation:

- p1 * p2 → dot product → 43
- 43 + 1 → 44

This works because the Point class overloaded + and *.

Definition of a Polymorphic Function

A function is **polymorphic** if it:

- Can be applied to arguments of different types
- Does not depend on the exact class of the arguments
- Only depends on the **operations** those arguments support

Example 2: front_and_back

```
def front_and_back(front):
    import copy
    back = copy.copy(front)
    back.reverse()
    print(str(front) + str(back))
```

Key Operations Used

- copy.copy
- reverse
- str

Applying It to a List

my_list = [1, 2, 3, 4]

front_and_back(my_list)

Output

[1, 2, 3, 4][4, 3, 2, 1]

This is expected because lists support:

- copy
- reverse
- str

Duck Typing Rule (Core of Python Polymorphism)

Python follows **duck typing**, which means:

"If it walks like a duck and quacks like a duck, it is a duck."

In programming terms:

If an object supports all the operations a function needs, the function can use that object.

Python does **not** check the type in advance—it simply tries the operations.

Making front_and_back Work with Point

We already have:

- copy.copy → works for all objects
- __str__ → already defined for Point

So we only need a reverse method in Point:

```
def reverse(self):
    (self.x, self.y) = (self.y, self.x)
```

Applying front_and_back to a Point

p = Point(3, 4)

front_and_back(p)

Output

(3, 4)(4, 3)

This is **unintentional polymorphism**—the function works with a type it was never designed for.

Intentional vs Unintentional Polymorphism

- **Intentional polymorphism**:
  - Designed explicitly (e.g., multadd)
- **Unintentional polymorphism**:
  - Discovered later (e.g., front_and_back with Point)

Both are powerful features of Python.

- Polymorphism allows functions to work with multiple types
- Python uses **duck typing**
- Type checking is based on **supported operations**, not class names
- Operator overloading enables polymorphism
- multadd is a polymorphic function
- front_and_back demonstrates unintentional polymorphism
- Polymorphism improves flexibility and reuse

5.3 Catching Exceptions

When a **runtime error** occurs in Python, the interpreter creates an **exception object**.
At that point:

- The normal flow of the program **stops**
- Python prints a **traceback**
- The traceback ends with a message describing **what went wrong**

Common Examples of Exceptions

1. Division by Zero

```
print(55 / 0)
```

Exception

ZeroDivisionError: integer division or modulo by zero

---

2. Accessing an Invalid List Index

```
a = []
print(a[5])
```

Exception

IndexError: list index out of range

3. Modifying an Immutable Object

```
tup = ("a", "b", "d", "d")
tup[2] = "c"
```

Exception

TypeError: 'tuple' object does not support item assignment

Understanding Error Messages

The last line of a traceback always has **two parts**:

1. **Exception type** (before the colon)
2. **Explanation** (after the colon)

Example:

ZeroDivisionError : integer division or modulo by zero

Why Catch Exceptions?

Sometimes an error is **expected** and should not crash the program.

Example:

- User enters a filename
- File may not exist
- We want to **handle the situation gracefully**

Using try and except

Basic Structure

```
try:
    # Code that may cause an exception
```

```
except SomeException:
    # Code that runs if that exception occurs
```

Example: Handling a Missing File

```
filename = input("Enter a file name: ")
try:
    f = open(filename, "r")
except FileNotFoundError:
    print("There is no file named", filename)
```

- If the file exists → program continues normally
- If the file does not exist → exception is caught, program does not crash

Full try Statement Structure

A try statement can have **four clauses**:

1. try
2. except
3. else
4. finally

Only try is mandatory.

Example with All Clauses

```
user_input = input('Type a number:')
try:
    user_input_as_number = float(user_input)
except ValueError:
    print('You did not enter a number.')
else:
    print('The square of your number is', user_input_as_number**2)
finally:
    print('Thank you')
```

Execution Flow

- **try** → Python attempts the operation
- **except** → runs only if the specified exception occurs
- **else** → runs only if no exception occurs
- **finally** → runs **always**, whether an exception occurs or not

Best Practice: Keep try Blocks Small

You should include **as little code as possible** inside a try block.

Why?

- Large try blocks can accidentally catch **unexpected errors**
- This can hide bugs and make debugging difficult

Handling Multiple Exceptions

A single try block can handle **different exceptions separately**:

```
try:
    with open(filename) as infile:
        content = infile.read()
except FileNotFoundError:
    print('The file does not exist.')
except PermissionError:
    print('You are not allowed to read this file.')
```

Each except handles a **specific error case**.

Avoid Bare except

```
except:
    print("Something went wrong")
```

This catches **all exceptions**, which is dangerous because:

- It can hide programming mistakes
- Makes debugging very hard

Recommendation: Always specify the exception type.

- Runtime errors create **exception objects**
- Traceback shows where the error occurred
- try / except is used to handle exceptions
- else runs only if no exception occurs
- finally always runs
- Handle specific exceptions whenever possible
- Avoid bare except
- Keep try blocks minimal

5.4 Raising Our Own Exceptions

Yes—**a Python program can deliberately raise its own exceptions**.
This is useful when the program detects an **error condition** and wants to signal that something is wrong instead of continuing with invalid data.

Why Raise Exceptions Yourself?

- To **validate input**
- To **stop execution** when data is invalid
- To **inform calling code** that an error occurred
- To keep code **clean and reliable**

Instead of returning incorrect values, we raise an exception and let the caller decide how to handle it.

Example: Validating User Input (get_age)

```
def get_age():
    age = int(input("Please enter your age: "))
    if age < 0:
        # Create a new instance of an exception
        my_error = ValueError("{0} is not a valid age".format(age))
        raise my_error
    return age
```

Step-by-Step Explanation

1. The user enters their age.
2. The input is converted to an integer.
3. If the age is **negative**, it is invalid.
4. A **ValueError object** is created with a meaningful message.
5. The exception is raised using raise.
6. The function **immediately exits**.

What Happens When an Exception Is Raised?

Assume the call sequence is:

A → B → C → D → get_age()

When raise executes:

- get_age() stops immediately
- Control returns to D
- Then to C, then to B, then to A
- This continues until:
    - A matching try ... except is found, **or**
    - Python prints a traceback and terminates the program

This process is called **unwinding the call stack**.

Example Execution

Valid Input

```
>>> get_age()
Please enter your age: 42
42
```

Invalid Input

```
>>> get_age()
Please enter your age: -2
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
  File "learn_exceptions.py", line 4, in get_age
```

raise ValueError("{0} is not a valid age".format(age))

ValueError: -2 is not a valid age

Understanding the Error Message

The traceback shows:

- Where the error occurred
- The **exception type** (ValueError)
- The **custom message** we provided

This makes debugging much easier.

Why Use ValueError?

ValueError is a **built-in exception** that is appropriate when:

- A value is of the correct type
- But has an **invalid value**

Age -2 is an integer, but not a valid age—so ValueError is correct.

Combining Creation and Raising (Common Practice)

Often, we combine these two steps:

raise ValueError("{0} is not a valid age".format(age))

This is equivalent to:

my_error = ValueError("{0} is not a valid age".format(age))

raise my_error

The textbook separates them initially to make the learning clearer.

- Programs can **raise their own exceptions**
- Use raise to signal an error condition
- Raising an exception immediately stops the function
- The exception travels up the call stack (stack unwinding)
- Built-in exceptions like ValueError should be used when appropriate
- Custom messages improve error clarity
- Exceptions can be handled by try ... except in calling code

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and student-focused resources for every semester.

Connect With Us:

www.azdocuments.in [Click Here]

@azdocuments [Click Here]

Join Our Student Community: [Click Here]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.